P1467R4
Extended floating-point types and standard names

Published Proposal, 2020-06-12

This version:
https://wg21.link/p1467r4

Issue Tracking:
Inline In Spec

Authors:
David Olsen (NVIDIA)
Michał Dominiak (NVIDIA)

Audience:
EWG, LEWG

Toggle Diffs:
☐ Hide deleted text

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

§ 1. Abstract

Allow implementations to define *extended floating-point types* in addition to the three standard floating-point types. Define rules for how the extended floating-point types interact with each other and with other types without changing the behavior of the existing standard floating-point types.

Specify the rules for type conversions, arithmetic conversions, promotions, narrowing conversions, and overload resolution in a way that strikes a balance between behaving like existing types and encouraging safe code. Specify the necessary library support, mostly additional overloads for functions that take floating-point arguments, for the extended floating-point types.

Define an optional set of `<cstdint>`-style type aliases for floating-point types matching specific, well-known floating-point layouts.

2. Revision history

2.1. R0 -> R1 (pre-Cologne)

Applied guidance from SG6 in Kona 2019:

1. Make the floating-point conversion rank not ordered between types with overlapping (but not subsetted) ranges of finite values. This makes the ranking a partial order.
2. Narrowing conversions are now based on floating-point conversion rank instead of ranges of finite values, which preservesthe current narrowing conversions relations between standard floating-point types; it also interacts favorably with the rank being a partial ordering.
3. Operations that deal with floating-point types whose conversion ranks are unordered are now ill-formed.
4. The relevant parts of the guidance have been applied to the library wording section as well.

Afterwards, applied suggestions from EWGI in Kona 2019 (this modifies some of the points above):

1. Apply the suggestion to make types where one has a wider range of finite values, but a lower precision than the other, unordered in their conversion rank, and therefore make operations that mix them ill-formed. The motivating example was IEEE-754 `binary16` and `bfloat16`; see Floating-point conversion rank for more details. This change also caused this paper to drop the term "range of finite values", since the modified semantics are better expressed in terms of sets of values of the types.
2. Add a change to narrowing conversions, to only allow exact conversions to happen.
3. Explicitly list parts of the language that are not changed by this paper; provide a more detailed analysis of the standard library impact.

§ 2.2. R1 -> R2 (pre-Belfast)

Changes based on feedback in Cologne from SG6, LEWGI, and EWGI. Further changes came from further development of the paper by the authors, especially overload resolution.

- Revised floating-point promotion rules. Removed all promotions other than `float` to `double`. Added wording for promoting values passed to varargs functions.
- Added the section on implicit conversions.
- Added the section on overload resolution.
- Added the section about feature test macros.
- Added the sections about the possibility of new library traits.
- Changed the wording for the `abs` function in the `<cmath>` section.
- Added constraints to the I/O streams overloads for `complex` to only support standard floating-point types.
- Added the section about possible changes to `<atomic>`.

§ 2.3. R2 -> R3 (pre-Prage)

Changes based on feedback in Belfast from EWG.

- Change the overload resolution rules, removing the rule that prefers one standard conversion over another based on conversion rank. Replace it with a rule that prefers one standard conversion over another only when the two types have the same representation.
- As a result of the overload resolution change, change floating-point promotion so that any type smaller than `double` promotes to `double`.
- Allow implicit conversions between pointer types that point to floating-point types with the same representation.

§ 2.4. R3 -> R4 (Summer 2020)

Merge P1468 into P1467. The two papers were separate proposals when first written. But over time they have become intertwined, with design decisions in one paper affecting the feasibility of the other. So the two papers are being merged into a single proposal in P1467R4.

Changes based on feedback in Prague from EWG, where the discussion was all about what the goals of the proposal should be. The group settled on a set of design decisions (see the poll results) that

strike a balance between the existing behavior of arithmetic types and a "safe by default" strategy.

Changes between P1467R3 and P1647R4:

- Add section § 4 C Compatibility
- Revert the rules for floating-point § 5.4 Promotion back to what they were in P1647R2, which is essentially unchanged from the current C++ standard. This was necessitated by changes to the overload resolution rules.
- Resolve the open issue of § 5.5 Implicit conversions. In R3, it was undecided if potentially lossy conversions should be implicit. EWG in Prague was strongly in favor of requiring lossy conversions to be explicit. The section on implicit conversions now reflects that guidance.
- Revert the rules for § 5.8 Overload resolution back to what they were in P1647R2, with a small fix to the proposed wording changes. Two alternate ideas for overload resolution are now listed.
- Withdraw the proposed change for § 5.9 Pointer conversions.

Changes to the content of P1468R3 as it was merged into P1647R4:

- Changed the proposed § 7.7 Literal suffixes to match what will be available in C2x.

§ 3. Motivation

16-bit floating-point support is becoming more widely available in both hardware (ARM CPUs and NVIDIA GPUs) and software (OpenGL, CUDA, and LLVM IR). Programmers wanting to take advantage of 16-bit floating-point support have been stymied by the lack of built-in compiler support for the type. A common workaround is to define a class type with all of the conversion operators and overloaded arithmetic operators to make it behave as much as possible like a built-in type. But that approach is cumbersome and incomplete, requiring inline assembly or other compiler-specific magic to generate efficient code.

The problem of efficiently using newer floating-point types that haven’t traditionally been supported can’t be solved through user-defined libraries. A possible solution of an implementation changing `float` to be a 16-bit type would be unpopular because users want support for newer floating-point types in addition to the standard types, and because users have come to expect `float` and `double` to be 32- and 64-bit types and have lots of existing code written with that assumption.

This problem is worth solving, and there is no viable solution under the current standard. So changing the core language in an extensible and backward-compatible way is appropriate. Providing a standard way for implementations to support 16-bit floating-point types will result in better code, more portable code, and wider use of those types.

While deciding what names to give to the 16-bit floating-point types, it was decided that C++ would benefit from having standard names for other larger floating-point types that are commonly used. Having names for specific floating-point formats allows users to more clearly specify their intent. If a user writes code that is designed for an IEEE 64-bit binary floating-point type, the code is more clear if it uses a name that is guaranteed to be IEEE 64-bit, and the failure mode is more immediate (a compilation error) if the code is ported to a system where an IEEE 64-bit type is not available. This part of the proposal is a revival, with modifications, of [N1703], which in 2013 proposed adding typedefs for fixed-layout floating-point types to both C and C++, but was not adopted by either language.

The motivation for the current approach of extended floating-point types comes from discussion of the previous paper [P0192]. That proposal's single new standard type of `short float` was considered insufficient, preventing the use of both IEEE-754 16-bit and `bfloat16` in the same application. When that proposal was rejected in November 2018, the current, more expansive, proposal was developed. It is not feasible to predict which floating-point types, or even how many different types, will be used in the future, so this proposal allows for as many types as the implementation sees fit.

§ 4. C Compatibility

The C standards committee, WG14, is proposing significant extensions to floating-point support in C as a new annex to the C2x standard. (The latest version is on an internal wiki and is not publicly available. An earlier version of the proposal is in [N2405].) The changes being worked on for C are compatible with the changes proposed for C++ in this proposal. Users will be able to write code that that uses IEEE floating-point types, include 16-bit binary, that compiles and behaves the same in both languages.

The C proposal adds optional types `_FloatN`, where *N* is 16, 32, 64, 128, or greater than 128 and divisible by 32. `_FloatN` is an IEEE binary floating-point type with the given size. These types should behave the same as the named aliases proposed below. (Except that C does not define a type for the non-IEEE `bfloat16` format.) The proposed usual arithmetic conversions when mixing different floating-point types are essentially the same in both languages.

There are two areas of divergence between the C and C++ proposals that are worth mentioning:

1. Names: The C proposal uses `_Float16`, `_Float32`, `_Float64`, and `_Float128` as keywords naming the IEEE types. This paper proposes type aliases in the `std` namespace. (See § 7.6 Names) Since C++ likes to have all its library names in namespace `std`, and C does not have namespace `std` at all, this area of divergence seems unavoidable. The C++ implementation could use `_Float16`, `_Float32`, etc. as the names of the extended floating-point types behind the

`std::` type aliases, allowing the use of the C names in both languages. But code that wants to work in both C and C++ and wants to maximize portability will need at least one `#if`, such as (with C++ names still subject to change):

```
#ifdef __cplusplus
  #include <stdfloat>
  using my_fp16_t = std::float16_t;
#else
  typedef _Float16 my_fp16_t;
#endif
```

2. Implicit conversions: In this C++ proposal, narrowing conversions between floating-point types have to be explicit. (See § 5.5 Implicit conversions) In the C proposal, conversions between floating-point types can be done implicitly, even when they are narrowing and potentially lossy. This will result in code using floating-point types that will compile as C but not as C++. While this divergence is unfortunate, it is acceptable because code using extended floating-point types that compiles successfully in both languages will behave the same in both languages.

The authors are comfortable with this proposal and the C proposal proceeding in parallel. The two proposals together move the two languages in compatible directions and do not cause unreasonable divergence. The authors are monitoring the progress of the C proposal and will raise issues with WG14 or WG21 (or both) if the proposals start to diverge as they evolve.

§ 5. Core language changes

§ 5.1. Things that aren’t changing

It is currently implementation-defined whether or not the floating-point types support infinity and NaN. That is not changing. That feature will still be implementation-defined, even for extended floating-point types.

The radix of the exponent of each floating-point type is currently implementation-defined. That is not changing. This paper will make it easier for the radix of extended floating-point types to be different from the radix of the standard types, allowing implementations to support decimal floating-point while the standard floating-point types remain binary floating-point types.

§ 5.2. Extended floating-point types

In addition to the three standard floating-point types, `float`, `double`, and `long double`, implementations may define any number of *extended floating-point types*, similar to how implementations may define extended integer types.

§ 5.2.1. Reasoning

The set of floating-point types that have hardware support is not possible to accurately predict years into the future. The standard needs to provide an extensible solution so that implementations can adapt to changing hardware without having to modify the standard.

§ 5.2.2. Wording

Modify 6.7.1 "Fundamental types" [**basic.fundamental**] paragraph 12:

> There are three *standard floating-point types*: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. There may also be implementation-defined *extended floating-point types*. The standard and extended floating-point types are collectively called *floating-point types*. The value representation of floating-point types is implementation-defined. [...]

§ 5.3. Conversion rank

Define *floating-point conversion rank* to mimic in some ways the existing integer conversion rank. Floating-point conversion rank is defined in terms of the sets of values that the types can represent. If the set of values of type `T` is a strict superset of the set of values of type `U`, then `T` has a higher conversion rank than `U`. If two types have the exact same sets of values, they still have different conversion ranks; see the wording below for the exact rules. If the sets of values of two types are neither a subset nor a superset of each other, then the conversion ranks of the two types are unordered. Floating-point conversion rank forms a partial order, not a total order; this is the biggest difference from integer conversion rank.

§ 5.3.1. Reasoning

Earlier versions of this proposal used the range of finite values to define conversion rank, and had the conversion rank be a total ordering. Discussions in SG6 in Kona 2019 pointed out that that definition resulted in undesirable interactions between IEEE `binary16` with 5-bit exponent and 10-bit mantissa, and `bfloat16` with 8-bit exponent and 7-bit mantissa. `bfloat16` has a much larger finite range, so it would have a higher conversion rank under the old rules. Mixing `binary16` and `bfloat16` in an arithmetic operation would result in the `binary16` value being converted to `bfloat16` despite the loss of three bits of precision. This implicit loss of precision was worrisome, so the definition of conversion rank was changed so that the usual arithmetic conversions between two floating-point values always preserves the value exactly.

For the purposes of conversion rank, infinity and NaN are treated just like any other values. If type `T` supports infinity and type `U` does not, then `U` can never have a greater conversion rank than `T`, even if `U` has a bigger range and a longer mantissa.

When an implementation supports both binary and decimal floating-point, the conversion ranks of a binary type and a decimal type will always be unordered, because neither type's set of values will be a subset of the other due to the different radixes. As a result, any arithmetic that mixes binary and decimal types will be ill-formed without explicit casts.

§ 5.3.2. Wording

Change the title of section 6.7.4 [**conv.rank**] from " ~~Integer conversion rank~~ " to " Conversion ranks ", but leave the stable name unchanged. Insert a new paragraph at the end of the subclause:

> Every floating-point type has a *floating-point conversion rank* defined as follows:
>
> - The rank of a floating point type `T` is greater than the rank of any floating-point type whose set of values is a proper subset of the set of values of `T`.
> - The rank of `long double` is greater than the rank of `double`, which is greater than the rank of `float`.
> - The rank of any standard floating-point type is greater than the rank of any extended floating-point type with the same set of values.
> - The rank of any extended floating-point type relative to another extended floating-point type with the same set of values is implementation-defined, but still subject to the other rules for determining the floating-point conversion rank.
> - For all floating-point types `T1`, `T2`, and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` has greater rank than `T3`.
>
> [*Note*: The conversion ranks of extended floating-point types `T1` and `T2` will be unordered if the set of values of `T1` is neither a subset nor a superset of the set of values of `T2`. This can happen when one type has both a larger range and a lower precision than the other. -- *end note*] [*Note*: The floating-point conversion rank is used in the definition of the usual arithmetic conversions ([expr.arith.conv]). -- *end note*]

§ 5.4. Promotion

Floating-point promotions are unchanged, except when passing an argument to a varargs function. When a function argument is bound to the ellipsis of a varargs function, any type whose conversion rank is smaller than that of `double` is promoted to `double`. In all other situations, only `float` is promoted to `double`.

§ 5.4.1. Reasoning

The overload resolution rules work best if there are no floating-point promotions, only standard conversions. For backward compatibility, `float` still promotes to `double`. But no other floating-point conversions are considered promotions.

But this rule prevents smaller extended floating-point types from being promoted to `double` when passed to a varargs function. Therefore, some wording is added to the ellipsis conversion rules to perform that promotion.

> Note: The current C floating-point proposal does not promote smaller floating-point types to `double` when calling varargs functions. This is an area where C and C++ should remain compatible, due primarily to `printf`. This issue will be discussed in the C floating-point study group in the near future. If the C floating-point proposal remains unchanged, then the proposed change to the ellipsis conversion rules will be withdrawn from this proposal, and there won't be any wording changes related to floating-point promotions.

5.4.2. Wording

No changes are necessary to 7.3.7 "Floating-point promotion" [conv.fpprom]

Add a new sentence just before the last sentence in paragraph 12 of 7.6.1.2 "Function call" [**expr.call**]:

> [...] If the argument has integral or enumeration type that is subject to the integral promotions (7.3.6), or a floating-point type that is subject to the floating-point promotion (7.3.7), the value of the argument is converted to the promoted type before the call. If the argument has floating-point type that is not subject to floating-point promotion, and if the argument type has a floating-point conversion rank ([conv.rank]) that is less than the rank of `double`, then the value of the argument is converted to `double` before the call. These promotions are referred to as the *default argument promotions*.

5.5. Implicit conversions

A conversion between two floating-point types, when at least one of the types is an extended floating-point type, is implicit only if the conversion is non-lossy, if the destination type can represent all values of the source type. Put another way, a conversion that might change the value is not a standard conversion.

5.5.1. Reasoning

The standard currently allows implicit conversions between any arithmetic types (except during brace init, when narrowing conversion rules apply), even if the conversion could result in a loss of information. This rule makes it too easy to write buggy code. Changing rules for existing types is not feasible because it would be a major breaking change. But the rules can be changed when types are used in new ways, as was done for brace init and narrowing conversions, or for new types, as is proposed here.

This was discussed in EWG in Prague, and there was consensus to limit implicit conversions for extended floating-point types. "Extended floating point types match the current C++ rules for conversions." 2-3-6-19-3 "Implicit conversions are only allowed if non-narrowing." 14-15-8-0-1

The conversion rules for standard floating-point types can't be changed without breaking existing code, so conversions from `double` to `float` and from `long double` to `double` or `float` will still be implicit.

§ 5.5.2. Wording

Modify section 7.3.9 "Floating-point conversions" [**conv.double**] as follows:

> A prvalue of floating-point type can be converted to a prvalue of another floating-point type with a higher conversion rank or with the same set of values, or a prvalue of standard floating-point type can be converted to a prvalue of another standard floating-point type . If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.
>
> The conversions allowed as floating-point promotions are excluded from the set of floating-point conversions.

In section 7.6.1.8 "Static cast" [**expr.static.cast**], add a new paragraph after paragraph 10 ("A value of integral or enumeration type can [...]"):

> A value of floating-point type can be explicitly converted to any other floating-point type. If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

> Note: A `static_cast` from a higher floating-point conversion rank to a lower conversion rank is already covered by [expr.static.cast] p7, which talks about inverses of standard conversions. The new paragraph is necessary to allow explicit conversions between types with unordered conversion ranks. The wording about what to do with the value is stolen from the floating-point conversions section [conv.double].

§ 5.6. Usual arithmetic conversions

The proposed usual arithmetic conversions for floating-point types are based on the floating-point conversion rank, similar to integer arithmetic conversions. But because floating-point conversions are a partial ordering, there may be some expressions where neither operand will be converted to the other's type. It is proposed that these situations are ill-formed.

§ 5.6.1. Example

> Note: In all the examples in this paper, `float` and `double` are IEEE 32-bit and 64-bit types, `std::floatN_t` is an extended floating-point type for IEEE *N*-bit, and `std::bfloat16_t` is `bfloat16`.

```
float f32 = 1.0;
std::float16_t f16 = 2.0;
std::bfloat16_t b16 = 3.0;
f32 + f16; // okay, f16 converted to "float", result type is "float"
f32 + b16; // okay, b16 converted to "float", result type is "float"
f16 + b16; // error, neither type can convert to the other via arithmetic c
```

§ 5.6.2. Wording

Modify section 7.4 Usual arithmetic conversions [**expr.arith.conv**] as follows:

1 Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:

(1.1) - If either operand is of scoped enumeration type ([dcl.enum]), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.

(1.2) - ~~If either operand is of type long double, the other shall be converted to long double.~~

(1.3) - ~~Otherwise, if either operand is double, the other shall be converted to double.~~

(1.4) - ~~Otherwise, if either operand is float, the other shall be converted to float.~~

(1.5) - Otherwise, if either operand has a floating-point type, the following rules shall be applied:

(1.1) - If both operands have the same type, no further conversion is needed.

(1.2) - Otherwise, if one of the operands has a type that is not a floating-point type, that operand shall be converted to the type of the operand with the floating-point type.

(1.3) - Otherwise, if the floating-point conversion ranks ([conv.rank]) of the types of the operands are ordered, then the operand with the type of the lower floating-point conversion rank shall be converted to the type of the other operand.

(1.4) - Otherwise, the expression is ill-formed.

(1.6) - Otherwise, the integral promotions ([conv.prom]) shall be performed on both operands.(59) Then the following rules shall be applied to the promoted operands:

(1.1) - If both operands have the same type, no further conversion is needed.

(1.2) - Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.

> (1.3) - Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.
>
> (1.4) - Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.
>
> (1.5) - Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.
>
> 2 If one operand is of enumeration type and the other operand is of a different enumeration type or a floating-point type, this behavior is deprecated (D.1).

§ 5.7. Narrowing conversions

A narrowing conversion is a conversion from a type with a higher floating-point conversion rank to a type with a lower conversion rank, or a conversion between two types with unordered conversion rank.

§ 5.7.1. Same representation

When two different floating-point types have the same representation, one of the types has a higher conversion rank than the other. Which means that a conversion between the two types will be a narrowing conversion in one of the directions even though the value will be preserved. For example, on some implementations, `double` and `long double` have the same representation, but `long double` always has a higher conversion rank than `double`, so a conversion from `long double` to `double` is considered a narrowing conversion.

An earlier version of this paper defined narrowing conversions in terms of sets of representable values, not in terms of conversion rank. With that definition, conversions between types with the same representation would never be a narrowing conversion. SG6 in Kona preferred using conversion rank over sets of values, so the proposal was changed to the current definition. One argument against the old definition was that it changed the behavior for standard floating-point types, as in the example of `double` and `long double` above.

It would be possible to have different rules for standard floating-point types and extended floating-point types, but the authors feel it is best to maintain consistency between standard and extended types, and to not change the behavior of standard types.

§ 5.7.2. Constant values

This proposal preserves the existing wording in [dcl.init.list] p7.2, "except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly)." A reasonable argument could be made that this constant value exception should not apply to extended floating-point types. But the authors are not in favor of that change. It would introduce an inconsistency between standard and extended types. It would cause `std::float16_t x{2.1};` to be a narrowing conversion because `2.1` cannot be represented exactly in binary floating-point representations.

§ 5.7.3. Wording

Modify the definition of narrowing conversions in 9.3.4 "List-initialization" [**dcl.init.list**] paragraph 7 item 2:

> - ~~from `long double` to `double` or `float`, or from `double` to `float`~~ from a floating-point type T to another floating-point type whose floating-point conversion rank is not greater than that of T , except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

§ 5.8. Overload resolution

When comparing conversion sequences that involve floating-point conversions, prefer conversions that are value-preserving, and prefer conversions to lower conversion ranks over conversions to higher conversion ranks.

5.8.1. Reasoning

With the proposed change to implicit conversions, preferring value-preserving conversions over lossy conversions comes for free, since overloads with lossy conversions won't be viable candidates (except when both types are standard floating-point types).

Preferring a conversion to a smaller type over a conversion to a larger type comes from the desire for a function call to be well-formed rather than ambiguous when there are multiple value-preserving conversions available.

```
void f(std::float32_t);
void f(std::float64_t);

f(std::float16_t(1.0)); // calls std::float32_t, due to smaller convers
f(float(2.0));          // calls std::float32_t, due to smaller convers
f(double(3.0));         // calls std::float64_t, only viable candidate
```

Achieving this behavior is not possible by tweaking the definitions of floating-point promotions and floating-point conversions. It requires a change to the overload resolution rules so that certain floating-point conversions are preferred over others.

This issue was debated in EWG in Prague, and these overload resolution rules received weak consensus. "Prefer smaller safe conversions over larger safe conversions in overload resolution." 3-14-10-0-7

5.8.2. Wording

In 12.3.3.2 "Ranking implicit conversion sequences" [**over.ics.rank**] paragraph 4, add a new bullet between (4.2) and (4.3):

- (4.2) A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.
- (4.3) A conversion from floating-point type `FP1` to floating-point type `FP2` is better than a conversion from `FP1` to floating-point type `FP3` if
  - (4.3.1) at least one of `FP1`, `FP2`, or `FP3` is an extended floating-point type,
  - (4.3.2) the set of values of `FP1` is a subset of the set of values of `FP2`, and
  - (4.3.3) `FP3` has greater floating-point conversion rank ([conv.rank]) than `FP2`, or `FP1` has greater floating-point conversion rank than `FP3`.
- ~~(4.3)~~ (4.4) If class `B` is derived directly or indirectly from class `A`, conversion of `B*` to `A*` is better than conversion of `B*` to `void*`, and conversion of `A*` to `void*` is better than conversion of `B*` to `void*`.

Note: (4.3.2) and the second half of (4.3.3) are necessary to correctly handle lossy conversions between standard floating-point types such as from `double` to `float`, which are still considered standard conversions and participate in overload resolution. (4.3.1) is necessary to preserve existing behavior when there are overloads for `float` and `long double` and the argument type is `double`.

5.8.3. Alternate proposals

The EWG poll about overload resolution did not have strong consensus, due to the significant number of neutral votes and strongly against votes. In light of that result, we present two alternate options for overload resolution rules. The authors are in favor of the proposed wording above, not the alternative proposals below.

5.8.3.1. Prefer same representation

The first alternative is to prefer conversions to types that have the same representation over safe conversions to bigger types. With this scheme:

```
void f(std::float32_t);
void f(std::float64_t);

f(std::float16_t(1.0)); // ambiguous
f(float(2.0));          // calls std::float32_t, because same representa
f(double(3.0));         // calls std::float64_t, only viable candidate
```

5.8.3.2. No change

The other alternative is to not change the overload resolution rules at all. There would be no disambiguation between standard conversions, so any call with multiple viable function overloads with no exact match would be ambiguous.

```
void f(std::float32_t);
void f(std::float64_t);

f(std::float16_t(1.0)); // ambiguous
f(float(2.0));          // ambiguous
f(double(3.0));         // calls std::float64_t, only viable candidate
```

5.9. Pointer conversions

The proposal of allowing implicit conversions between pointers to two different floating-point types that have the same representation was voted down by EWG in Prague, so it has been withdrawn from this proposal. Allowing the implicit pointer conversions would have eased the transition from using the standard floating-point types to the new named floating-point types. But it complicated the language in a non-obvious way, and the group decided that the benefit was not worth the cost.

§ 5.10. Feature test macro

> ISSUE 1 Should there be a feature test macro to indicate that the implementation supports at least one extended floating-point type?

Implementations could support extended floating-point types without supporting any of the aliases for well-known layouts. It might be useful to have a feature test macro that indicates support for extended floating-point types listed in 15.11 [**cpp.predefined**]. But it would likely have to be one of the conditionally-defined macros, and not listed in Table 17, since a conforming compiler might choose to not define any extended floating-point types. If the macro is defined, it would not indicate which extended floating-point types are supported, only that there exists at least one extended floating-point type in the implementation. The authors believe that such a feature test macro would not be useful, but would like SG10 to confirm that decision.

§ 6. Library changes

Making extended floating-point types easy to use does not require introducing any new names to the standard library. But it does require adding new overloads or new template specializations in several places. Some of the extended floating-point types will have standard names. Those new names are covered in § 7 Type aliases.

To handle I/O of extended floating-point types, changes are proposed to `<charconv>` and `<format>`, but not to `<iostream>` or `<cstdio>`.

Implementations will have to change `std::numeric_limits` and `std::is_floating_point` to give correct answers for extended floating-point types. The existing wording in the standard already covers that (by referring to all *floating-point types* without listing them explicitly), so no wording changes are needed.

Most of the standard functions that operate on floating-point types need wording changes to add overloads or template specializations for the extended floating-point types. These classes and functions are in `<cmath>`, `<complex>`, and `<atomic>`.

No changes are proposed to the following parts of the standard library:

- `<cfloat>`: The header `<cfloat>` provides macros describing some of the properties of the standard floating-point types. The use of macros does not extend very well to extended floating-point types with implementation-specific names. Users should use `std::numeric_limits` rather than macros from `<cfloat>` to query the properties of extended floating-point types.

- The `printf` and `scanf` families of functions: There is no practical way to add specifiers for implementation-specific types with implementation-specific names.
- The `strtod` and `stod` families of functions: With different names for each floating-point type (which for `strtod` was inherited from C), that scheme doesn't work well for extended floating-point types.
- I/O streams: There is currently no support for extended integer types. Correctly supporting extended floating-point types larger than `long double` would require ABI-breaking changes to `num_get` and `num_put`.
- The `std::to_string` family of functions: They are defined in terms of `snprintf`, which will not support extended floating-point types.
- `<random>`: [rand.req] states that certain template arguments have to be `float`, `double`, or `long double`. The wording could be changed to allow any floating-point type, but `<random>` does not support extended integral types, so we are not proposing that it support extended floating-point types either.

WG14 is working on adding optional support for additional floating-point types in an annex to C2x. (See § 4 C Compatibility.) If those changes to the C standard library land in C2x, then C++ users will eventually see support for some of C++'s extended floating-point types through macros defined in `<cfloat>` and conversion functions in `<cstdlib>`. This proposal is not suggesting identical changes ahead of C2x in these areas. The changes will have to come to C++ through C2x.

§ 6.1. Possible new names

While no new names need to be added to the standard library for extended floating-point types to be useful, there are some new things that could be useful. The authors are undecided if these are useful enough to be worth adding, and would appreciate LEWG feedback on the matter.

§ 6.1.1. Standard/extended floating-point traits

`std::is_floating_point_v<T>` is true for both standard and extended floating-point types. Should the standard also provide `std::is_standard_floating_point` and/or `std::is_extended_floating_point`? Will users need to distinguish between standard and extended types often enough that `std::is_same_v<T, float> || std::is_same_v<T, double> || std::is_same_v<T, long double>` becomes too unwieldy?

> ISSUE 2 Should the new type traits `std::is_standard_floating_point` and/or `std::is_extended_floating_point` be introduced?

6.1.2. Conversion rank trait

Should there be a type trait that reports whether or not one floating-point type has a higher conversion rank than another? This could be useful when writing function templates to figure out which conversions between different floating-point types are safe. See the constructors for `std::complex` as an example of where this trait would be useful.

> ISSUE 3 Should a new type trait be introduced that can be used to query the floating-point conversion rank relationship?

6.2. `<charconv>`

Add overloads for all extended floating-point types for the functions `to_chars` and `from_chars`.

6.2.1. Wording

Add a new paragraph to the beginning of 20.19.1 "Header `<charconv>` synopsis" [**charconv.syn**], before the start of the synopsis:

> When a function has a parameter of type *`integral`*, the implementation provides overloads for all signed and unsigned integer types and `char` as the parameter type. When a function has a parameter of type *`floating-point`*, the implementation provides overloads for all floating-point types as the parameter type.

Change the header synopsis in [**charconv.syn**] as follows:

```
  to_chars_result to_chars(char* first, char* last, see-belowintegral va
  to_chars_result to_chars(char* first, char* last, floatfloating-point
- to_chars_result to_chars(char* first, char* last, double value);
- to_chars_result to_chars(char* first, char* last, long double value);
  to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt);
- to_chars_result to_chars(char* first, char* last, double value, chars_
- to_chars_result to_chars(char* first, char* last, long double value, c
  to_chars_result to_chars(char* first, char* last, floatfloating-point
                           chars_format fmt, int precision);
- to_chars_result to_chars(char* first, char* last, double value,
-                          chars_format fmt, int precision);
- to_chars_result to_chars(char* first, char* last, long double value,
-                          chars_format fmt, int precision);

  // ...

  from_chars_result from_chars(const char* first, const char* last,
                               see-belowintegral& value, int base = 10);

  from_chars_result from_chars(const char* first, const char* last, floa
                               chars_format fmt = chars_format::general)
- from_chars_result from_chars(const char* first, const char* last, doub
-                              chars_format fmt = chars_format::general)
- from_chars_result from_chars(const char* first, const char* last, long
-                              chars_format fmt = chars_format::general)
```

In 20.19.2 "Primitive numeric output conversion" [**charconv.to.chars**], leave the first three paragraphs unchanged, but modify the rest of the section as follows:

```
to_chars_result to_chars(char* first, char* last, see belowintegral valu
```

Requires *Expects* : `base` has a value between 2 and 36 (inclusive).

Effects: The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits inthe range 10..35 (inclusive) are represented as lowercase characters `a`..`z`. If `value` isless than zero, the representation starts with `'-'`.

Throws: Nothing.

Remarks: [*Note:* The implementation shall provide provides overloads for all signed and unsigned integer types and `char` as the type of the parameter `value`. *- end note*]

```
to_chars_result to_chars(char* first, char* last, floatfloating-point va
to_chars_result to_chars(char* first, char* last, double value);
to_chars_result to_chars(char* first, char* last, long double value);
```

Effects: `value` is converted to a string in the style of `printf` in the "C" locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.

Throws: Nothing.

[*Note*: The implementation provides overloads for all floating-point types as the type of the parameter `value`. *- end note*]

```
to_chars_result to_chars(char* first, char* last, floatfloating-point va
to_chars_result to_chars(char* first, char* last, double value, chars_fo
to_chars_result to_chars(char* first, char* last, long double value, cha
```

Requires *Expects* : `fmt` has the value of one of the enumerators of `chars_format`.

Effects: `value` is converted to a string in the style of `printf` in the "C" locale.

Throws: Nothing.

[*Note*: The implementation provides overloads for all floating-point types as the type of the parameter `value`. *- end note*]

```
to_chars_result to_chars(char* first, char* last, float floating-point va
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, double value,
                         chars_format fmt, int precision);
to_chars_result to_chars(char* first, char* last, long double value,
                         chars_format fmt, int precision);
```

Requires *Expects* : `fmt` has the value of one of the enumerators of `chars_format`.

Effects: `value` is converted to a string in the style of `printf` in the "C" locale with the given precision.

Throws: Nothing.

[*Note*: The implementation provides overloads for all floating-point types as the type of the parameter `value`. - *end note*]

See also: ISO C 7.21.6.1

Modify 20.19.3 "Primitive numeric input conversion" [**charconv.from.chars**] as follows:

1 All functions named `from_chars` analyze the string `[first, last)` for a pattern, where `[first, last)` is required to be a valid range. If no characters match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `errc::invalid_argument`. [*Note*: If the pattern allows for an optional sign, but the string has no digit characters following the sign, no characters match the pattern. — *end note*] Otherwise, the characters matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first character not matching the pattern, or has the value `last` if all characters match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `errc::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest`, and the member `ec` is value-initialized.

```
from_chars_result from_chars(const char* first, const char* last,
                             see belowintegral& value, int base = 10);
```

2 ~~*Requires*~~ *Expects* : `base` has a value between 2 and 36 (inclusive).

3 *Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale for the given nonzero base, as described for `strtol`, except that no `"0x"` or `"0X"` prefix shall appear if the value of `base` is 16, and except that `'-'` is the only sign that may appear, and only if `value` has a signed type.

4 *Throws*: Nothing.

5 ~~*Remarks:*~~ [*Note*: The implementation ~~shall provide~~ provides overloads for all signed and unsigned integer types and `char` as the referenced type of the parameter `value`. *- end note*]

```
from_chars_result from_chars(const char* first, const char* last, floatf
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, double
                             chars_format fmt = chars_format::general);
from_chars_result from_chars(const char* first, const char* last, long d
                             chars_format fmt = chars_format::general);
```

6 ~~*Requires*~~ *Expects* : `fmt` has the value of one of the enumerators of `chars_format`.

7 *Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale, as described for `strtod`, except that

(7.1) - the sign `'+'` may only appear in the exponent part;

(7.2) - if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;

(7.3) - if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and

(7.4) - if `fmt` is `chars_format::hex`, the prefix `"0x"` or `"0X"` is assumed. [*Example*: The string `0x123` is parsed to have the value `0` with remaining characters `x123`. - *end example*]

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

8 *Throws*: Nothing.

9 [*Note*: The implementation provides overloads for all floating-point types as the referenced type of the parameter `value`. - *end note*]

See also: ISO C 7.22.1.3, 7.22.1.4

§ 6.3. `<format>`

Change `std::format` to support extended floating-point types.

§ 6.3.1. Wording

... to be determined ...

§ 6.4. `<cmath>`

Add overloads for extended floating-point types to the functions in `<cmath>`. It is expected that this will be the most used part of the library changes.

§ 6.4.1. Wording

Modify 26.8.1 "Header `<cmath>` synopsis" [**cmath.syn**] paragraph 2 as follows:

For each set of overloaded functions within `<cmath>`, with the exception of `abs`, there shall be additional overloads sufficient to ensure:

- ~~1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (6.7.1) corresponding to `double` parameters are effectively cast to `long double`.~~
- ~~2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.~~
- ~~3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.~~
- 1. If any argument corresponding to a `double` parameter has floating-point type, then all arguments of arithmetic type ([basic.fundamental]) corresponding to `double` parameters are effectively cast to the floating-point type with the highest floating-point conversion rank ([conv.rank]) among the types of such floating-point arguments. If two such floating-point arguments have types whose conversion rank is unordered, the program is ill-formed.
- 2. Otherwise, all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.

[*Note*: `abs` is exempted from these rules in order to stay compatible with C. -- *end note*]

Modify section 26.8.2 "Absolute values" [**c.math.abs**] as follows:

1 [*Note*: The headers `<cstdlib>` and `<cmath>` declare the functions described in this subclause. — *end note*]

```
int abs(int j);
long int abs(long int j);
long long int abs(long long int j);
float abs(float j);
double abs(double j);
long double abs(long double j);
```

2 *Effects*: The `abs` functions that take integer arguments have the semantics specified in the C standard library for the functions `abs`, `labs`, and `llabs` , `fabsf`, `fabs`, and `fabsl` .

3 *Remarks*: If `abs()` is called with an argument of type `X` for which `is_unsigned_v<X>` is `true` and if `X` cannot be converted to `int` by integral promotion, the program is ill-formed. [*Note*: Arguments that can be promoted to `int` are permitted for compatibility with C. — *end note*]

```
floating-point abs(floating-point x);
```

4 *Returns*: The absolute value of `x`.

5 *Remarks*: The implementation provides overloads for all floating-point types as the type of parameter `x`, with the same floating-point type as the return type.

See also: ISO C 7.12.7.2, 7.22.6.1

§ 6.5. `<complex>`

Make `std::complex<T>` be well-defined when `T` is an extended floating-point type. The explicit specializations of `std::complex<T>` are removed. The only differences between the explicit specializations was the explicit-ness of the constructors that take a complex number of a different type. This behavior is incorporated into the main template through `explicit(bool)`.

§ 6.5.1. Wording

Modify 26.4 "Complex numbers" [**complex.numbers**] paragraph 2 as follows:

> The effect of instantiating the template `complex` for any type ~~other than `float`, `double`, or `long double`~~ that is not a floating-point type is unspecified. The specializations ~~`complex<float>`, `complex<double>`, and `complex<long double>`~~ of `complex` for floating-point types are literal types ([basic.types]).

Delete the explicit specializations from 26.4.1 "Header `<complex>` synopsis" [**complex.syn**]:

```
namespace std {
  // 26.4.2, class template complex
  template class complex;

  // 26.4.3, specializations
  template<> class complex;
  template<> class complex;
  template<> class complex;

  // ...
```

In 26.4.2 "Class template `complex`" [**complex**], modify the synopsis of the constructors as follows:

```
constexpr complex(const T& re = T(), const T& im = T());
constexpr complex(const complex&) = default;
template<class X> constexpr explicit(see below) complex(const complex<X>
```

Remove section 26.4.3 "Specializations" [**complex.special**] in its entirety.

In 26.4.4 "Member functions" [**complex.members**], add the following after paragraph 2:

```
template<class X> constexpr explicit(see below) complex(const complex<X>
```

Ensures: `real() == other.real() && imag() == other.imag()`.

Remarks: The expression inside `explicit` evaluates to false if and only if the floating-point conversion rank of `T` is greater than the floating-point conversion rank of `X`.

In 26.4.6 "Non-member operations" [**complex.ops**], change the streaming operators as follows:

```
template<class T, class CharT, class traits>
  basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&
```

Constraints: T is a standard floating-point type.

~~*Requires*~~ *Expects* : The input values ~~shall be~~ are convertible to `T`.

Effects: Extracts a complex number `x` of the form: `u`, `(u)`, or `(u,v)`, where `u` is the real part and `v` is the imaginary part (29.7.4.2).

If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios::failure` (29.5.5.4)).

Returns: `is`.

Remarks: This extraction is performed as a series of simpler extractions. Therefore, the skipping of whitespace is specified to be the same for each of the simpler extractions.

```
template<class T, class charT, class traits>
  basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&
```

Constraints: T is a standard floating-point type.

Effects: Inserts the complex number `x` ...

Modify 26.4.9 "Additional overloads" [**cmplx.over**] paragraphs 2 and 3 as follows:

The additional overloads shall be sufficient to ensure:

- ~~If the argument has type `long double`, then it is effectively cast to `complex<long double>`.~~
- ~~Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.~~
- ~~Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.~~
- If the argument has a floating-point type `T`, then it is effectively cast to `complex<T>`.
- Otherwise, if the argument has integer type, then it is effectively cast to `complex<double>`.

Function template `pow` shall have additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

- ~~If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.~~
- ~~Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.~~
- ~~Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.~~
- If one argument is of type `T1` or `complex<T1>` and the other argument is of type `T2` or `complex<T2>` where `T1` and `T2` are both floating-point types:
  - If the floating-point conversion ranks ([conv.rank]) of `T1` and `T2` are different and unordered, the program is ill-formed.
  - Otherwise, if `T1` has greater floating-point conversion rank than `T2`, then both arguments are effectively cast to `complex<T1>`.
  - Otherwise, both arguments are effectively cast to `complex<T2>`.
- Otherwise, if the other argument has integer type, it is effectively cast to `complex<T>`.

Note: No literal suffixes are defined for complex numbers of extended floating-point types. Subclause [complex.literals] is unchanged.

ISSUE 4 Should literal suffixes be defined for complex numbers of extended floating-point types with standard names, similar to the non-complex suffixes?

§ 6.6. `<atomic>`

Change the wording so that the specializations of `std::atomic` for floating-point types apply to all floating-point types, not just the standard floating-point types listed.

The specializations of `std::atomic` for integral types are not required to include specializations for all extended integral types, only for the extended types that are used in `<cstdint>`. It would be reasonable for this proposal to adopt a similar approach.

> ISSUE 5 Should `std::atomic` have specializations for all floating-point types, or only for extended floating-point types with well-known aliases?

§ 6.6.1. Wording

This wording assumes that `std::atomic` supports all extended floating-point types. The wording would be different if it only needed to support named aliases.

Modify 31.8.3 "Specializations for floating-point types" [**atomics.types.float**] paragraph 1 as follows:

> There are specializations of the `atomic` class template for ~~the~~ all floating-point types ~~`float`, `double`, and `long double`~~ . For each such type *`floating-point`*, the specialization `atomic<`*`floating-point`*`>` provides additional atomic operations appropriate to floating-point types.

§ 6.7. Feature test macro

No feature test macro is being proposed for the library changes in this section. These library changes would be covered by the core language feature test macro, if there is one.

§ 7. Type aliases

This paper introduces type aliases for several fixed-layout floating-point types. Each alias will be defined only if a type with that layout is supported by the implementation, similar to the `intN_t` and `uintN_t` aliases.

§ 7.1. Header name

The type aliases proposed here do not fit neatly into any existing header. So we are offering up two possibilities for new header names, neither of which we are thrilled with: `<fixed_float>` and `<stdfloat>`. We are open to other names for the header and to arguments that the type aliases should be added to an existing header.

> ISSUE 6 What new or existing header should the type aliases go into?

§ 7.2. Supported formats

We propose aliases for the following layouts:

- [IEEE-754-2008] `binary16` - IEEE 16-bit.
- [IEEE-754-2008] `binary32` - IEEE 32-bit.
- [IEEE-754-2008] `binary64` - IEEE 64-bit.
- [IEEE-754-2008] `binary128` - IEEE 128-bit.
- `bfloat16`, which is `binary32` with 16 bits of precision truncated; see [bfloat16].

`binary32` and `binary64` are the most widely used floating-point types, and are the formats that `float` and `double` have in most implementations. `binary16` is becoming more widely used; see this paper's motivation for details. `binary128` has hardware support in IBM POWER P9 chips. `bfloat16` is used in Google's TPUs and in TensorFlow and has hardware support in NVIDIA's latest GPUs.

The most widely used format that is not in this list is X87 80-bit. Even though there is hardware support for this format in all current x86 chips, it is used most often because it is the largest type available, not because users specifically want that format.

§ 7.3. Aliasing standard types

This has turned out to be the most contentious issue with the type aliases, with strong opinions on both sides. In Cologne, SG6 and LEWGI voted in favor of allowing aliasing of standard types, while EWGI was strongly against the idea. After the Cologne meeting, the authors decided that prohibiting aliases of standard types was the better choice. EWG discussed the issue in Prague and there was very strong consensus for the authors' position. "The new floatX_t types aren't aliases for float / double / long double, they are independent types." 23-13-0-2-0

The header `<cstdint>` defines integer type aliases for certain integer types, such as `std::int32_t` and `std::int64_t`. These are similar in many ways to the aliases proposed here. The types in `<cstdint>` are allowed to alias standard integer types. That has resulted in compilation errors when users try to create an overload set with both standard types and fixed-layout aliases, such as:

```
int bit_count(int x) { /* ... */ }
int bit_count(std::int32_t x) { /* ... */ }
```

If aliasing of standard types is allowed for the floating-point type aliases, then similar compilation errors will likely result:

```
int get_exponent(double x) { /* ... */ }
int get_exponent(std::float64_t x) { /* ... */ }
```

This is the strongest argument against allowing aliasing of standard types. People who don't find this argument persuasive point out that users should not create overload sets with both standard types and fixed-layout type aliases. An overload set should contain just the standard floating-point types or just the fixed-layout types, but not both. The example above that fails to compile is considered poor design and should not be encouraged.

(The arguments about overload sets apply equally to explicit template specializations.)

Not allowing the aliasing of standard types imposes an implementation burden. If aliasing were allowed, then implementations that don't define any extended floating-point types could define some of the aliases with a little bit of library code that boils down to something like:

```
namespace std {
  using float32_t = float;
  using float64_t = double;
}
```

But when aliasing is not allowed, implementations have to support extended floating-point types in at least the compiler front end, which is not a trivial task. There is also a burden on the name mangling

ABI, which will have to define how to encode these extended floating-point types.

The authors feel that the burden on users of allowing aliasing of standard types is greater than the burden on implementers of not allowing such aliasing.

(This issue of aliasing of standard types is tightly bound to the overload resolution rules (§ 5.8 Overload resolution) for extended floating-point types. If the overload resolution rules are not changed, then having `std::float64_t` be an alias of an extended floating-point type rather than an alias of `double` will cause the following code to not compile:

```
void f(std::float32_t);
void f(std::float64_t);
void g(double x) {
  f(x); // error - ambiguous call without overload resolution changes
}
```

If that code doesn't compile, that would be a bigger burden on users than not being able to overload on both `double` and `std::float64_t`.)

§ 7.4. Layout vs. behavior

The IEEE-conforming type aliases must have the specified IEEE layout and should have the required behavior. For the four IEEE-conforming type aliases, `std::numeric_limits<T>::is_iec559` is true.

§ 7.5. Feature test macros

Since implementations may choose to support (or not) each of the fixed-layout aliases individually, there should be a separate test macro for detecting each of the type aliases. The names of the test macros would be derived from whichever type alias names we settle on. (The authors are not thrilled with introducing so many new test macros, but they have yet to come up with a better idea.)

> ISSUE 7 How should feature test macros be handled for this feature?

§ 7.6. Names

We are proposing several different naming schemes for fixed-layout type alias, and are open to other suggested naming schemes. In committee discussions so far, no set of names has emerged as the favorites. The authors have whittled proposed names down to what they feel are the three best choices, and are comfortable leaving it up to the committee to choose between those.

§ 7.6.1. `floatX_t`

- `std::float16_t`
- `std::float32_t`
- `std::float64_t`
- `std::float128_t`
- `std::bfloat16_t`

This is the simplest of all the options being presented. It is the naming scheme used by Boost.Math's fixed-layout floating-point types.

Nothing in the names of the IEEE aliases implies that they are in fact IEEE binary formats. Additionally, `float16_t` and `bfloat16_t` are similar enough that we aren't fully comfortable using these names.

§ 7.6.2. `fp::binaryX_t`

- `std::fp::binary16_t`
- `std::fp::binary32_t`
- `std::fp::binary64_t`
- `std::fp::binary128_t`
- `std::fp::bfloat16_t`

The namespace `fp` makes it more obvious that these types are floating-point types, assisting in the recognition of `binary16` as an [IEEE-754-2008] format. A using namespace directive can be used to avoid repeating `std::fp::` everywhere.

The drawbacks of this approach are that it introduces a new namespace with a very small purpose, and that `std::fp::bloat16_t` is somewhat redundant with two different floating-point indications (`fp`

and the `float` in `bfloat16_t`).

7.6.3. `fp_binaryX_t`

- `std::fp_binary16_t`
- `std::fp_binary32_t`
- `std::fp_binary64_t`
- `std::fp_binary128_t`
- `std::fp_bfloat16_t`

This is a slight modification of the previous scheme, which trades the nested namespace for an `fp_` prefix. The advantages and disadvantages are similar.

7.7. Literal suffixes

The types with standard-defined names should also have standard literal suffixes, similar to what is proposed in [P1280]. The suffixes for the IEEE types match what is being proposed for C2x. An implementation would define literal suffixes only for types supported by that implementation. The declarations of the literals might look something like this:

```
namespace std {
  inline namespace literals {
  inline namespace float_literals {
    constexpr float16_t operator""f16(const char *);
    constexpr float32_t operator""f32(const char *);
    constexpr float64_t operator""f64(const char *);
    constexpr float128_t operator""f128(const char *);
    constexpr bfloat16_t operator""bf16(const char *);
  }
  }
}
```

§ References

§ Informative References

[BFLOAT16]
bfloat16 floating-point format. URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

[IEEE-754-2008]
IEEE Standard for Floating-Point Arithmetic. 29 August 2008. URL: http://ieeexplore.ieee.org/servlet/opac?punumber=4610933

[N1703]
Paul A. Bristow; Christopher Kormanyos; John Maddock. Floating-Point Typedefs Having Specified Widths. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1703.pdf

[N2405]
Annex X: IEC 60559 interchange and extended types. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2405.pdf

[P0192]
Michał Dominiak; et al. `short float` and fixed-size floating point types. URL: https://wg21.link/P0192

[P1280]
Isabella Muerte. Integer Width Literals. URL: https://wg21.link/P1280

§ Issues Index

ISSUE 1 Should there be a feature test macro to indicate that the implementation supports at least one extended floating-point type? ↵

ISSUE 2 Should the new type traits `std::is_standard_floating_point` and/or `std::is_extended_floating_point` be introduced? ↵

ISSUE 3 Should a new type trait be introduced that can be used to query the floating-point conversion rank relationship? ↵

ISSUE 4 Should literal suffixes be defined for complex numbers of extended floating-point types with standard names, similar to the non-complex suffixes? ↵

ISSUE 5 Should `std::atomic` have specializations for all floating-point types, or only for extended floating-point types with well-known aliases? ↵

ISSUE 6 What new or existing header should the type aliases go into? ↵

ISSUE 7 How should feature test macros be handled for this feature? ↵